

CLI Agent AB Mode and Gate Semantics Profile v0.1

Dry-run, apply, enforce, confirm phrases, arm flags, local exceptions, and fail-closed execution semantics for C-Governed CLI Agent Mesh operations

Status	Draft normative profile v0.1
Date	2026-05-17
Package	C-Governed CLI Agent Mesh
Layer	c = a + b / Agent Governance / AB Mode / Gate Semantics / SYNAPS / Codex Bridge / Witness / Conformance
Document class	gate-semantics profile / implementation-readiness artifact / control-layer companion
Assertion class	C-A10 control-layer artifact; C-A7 where witness, hash, ledger, or verification claims are made
Distribution default	public-safe after hygiene review; implementation notes may reference restricted local code only by abstract gate class
Primary parent documents	C-Governed_CLI_Agent_Mesh_Protocol_v0_1.md; CLI_Agent_Task_Contract_Schema_v0_1.md; CLI_Agent_Permission_and_Capability_Model_v0_1.md; CLI_Agent_Handshake_Profile_v0_1.md; CLI_Agent_Sandbox_Worktree_Profile_v0_1.md; CLI_Agent_Witness_Event_Profile_v0_1.md; CLI_Agent_Memory_Gate_Profile_v0_1.md; CLI_Agent_Rollback_and_Freeze_Profile_v0_1.md; CLI_Agent_Quorum_and_Review_Profile_v0_1.md; CLI_Agent_Executor_Reviewer_Separation_v0_1.md; CLI_Agent_Public_Redaction_Profile_v0_1.md; CLI_Agent_Raw_Evidence_Sidecar_Profile_v0_1.md; CLI_Agent_Release_and_Implementation_Readiness_Gate_v0_1.md; CLI_Agent_JSON_Schema_Extraction_Plan_v0_1.md; CLI_Agent_Conformance_Fixture_Pack_v0_1.md
Primary object family	CLI_AGENT_AB_MODE_DECLARATION, CLI_AGENT_GATE_SEMANTICS_RECORD, CLI_AGENT_CONFIRM_PHRASE_RECORD, CLI_AGENT_ARM_STATUS_RECORD, CLI_AGENT_DRY_RUN_APPLY_RECORD, CLI_AGENT_LOCAL_AB_EXCEPTION_RECORD, CLI_AGENT_GATE_EVALUATION_RECORD
Canonical schema version	cli-agent-ab-mode-gate-semantics-0.1
Primary boundary	A/B, dry_run, apply, enforce, confirm phrases, arm flags, runner gates, persistent gates, and local module exceptions must have explicit semantics. No implementation may infer durable authority from a raw AB_MODE=B flag alone.

Contents

0. Executive definition	4
1. Purpose	4
2. Non-goals	5
3. Corpus bridge set	6
3.1 Explicit bridge: $c = a + b$	6
3.2 Quiet bridge I: information theory	6
3.3 Quiet bridge II: cybernetics	6
3.4 Earth paragraph	6
4. Core terminology	7
5. Global AB semantics	8
5.1 Mode A : observe / dry-run / plan	8
5.2 Mode B : controlled apply / enforce	8
6. Gate hierarchy	9
6.1 Gate precedence	9
7. Confirm phrase classes	10
8. Arm and second-arm rules	11
8.1 Required fail-closed cases	11
8.2 Why second arms exist	12
9. Local AB exception rules	12
9.1 Default rule	12
9.2 Local inversion rule	12
9.3 Read-only same-mode rule	13
9.4 Deprecated or unknown flags	13
10. Required object records	13
10.1 CLI_AGENT_AB_MODE_DECLARATION	13
10.2 CLI_AGENT_GATE_SEMANTICS_RECORD	14
10.3 CLI_AGENT_CONFIRM_PHRASE_RECORD	14
10.4 CLI_AGENT_GATE_EVALUATION_RECORD	14
10.5 CLI_AGENT_LOCAL_AB_EXCEPTION_RECORD	15
11. Witness, memory, and evidence interaction	15
11.1 Witness	15
11.2 Memory	15
11.3 Raw evidence	16
12. Implementation mapping to SYNAPS / Codex bridge	16
13. Conformance rules	16
AB-CONF-001 — Unknown mode fails to A	16
AB-CONF-002 — Dry-run writes nothing	17

AB-CONF-003 — Apply without confirm fails closed.....	17
AB-CONF-004 — Runner requires stronger gate	17
AB-CONF-005 — Persistent requires separate persistent gate.....	17
AB-CONF-006 — Persistent and runner must not silently combine.....	18
AB-CONF-007 — Local inversion must be declared	18
AB-CONF-008 — Memory and auto-ingest defaults are off.....	18
AB-CONF-009 — Confirm phrase class cannot be upgraded silently.....	18
AB-CONF-010 — B does not bypass public/restricted split.....	18
14. Red-line failures	19
15. Required integration with Local Checker.....	19
16. Required integration with Schema Object Registry	20
17. Required integration with Open Issues and Contradiction Register	20
18. Public redaction boundary.....	20
19. Status claims allowed by this profile	21
20. Minimal implementation-readiness checklist	21
21. Canonical short rules	22
22. Closing note	22

0. Executive definition

CLI Agent AB Mode and Gate Semantics Profile defines the common meaning of A/B operational modes and gate transitions for the C-Governed CLI Agent Mesh.

The profile exists because the implementation substrate already uses several related but distinct control concepts:

```
AB mode
dry_run flag
apply flag
confirm phrase
environment arm flag
second arm flag
runner gate
persistent gate
read-only sandbox constraint
memory=off default
auto_ingest=false default
```

These concepts must not be collapsed.

Compact formula:

```
A is observe.
B is not permission.
Apply is not authority.
A confirm phrase is not a contract.
A gate passes only when all required conditions pass.
Unknown means fail closed.
```

The profile establishes a common rule:

```
Global CGAM semantics:
A = observe / plan / dry-run / no durable mutation.
B = controlled apply / enforce / durable mutation allowed only after all gates pass.
```

But it also records an important implementation reality:

```
Some local modules may use local A/B flags differently.
Any local inversion or no-op-on-B behavior must be explicitly declared, tested, and blocked from being
treated as global authority.
```

1. Purpose

CLI agents and supporting services can now perform material operations:

```
move files
mark quarantine state
enqueue handoffs
create request objects
run local commands
write ledgers
```

```

promote mailbox items
observe reports
create packages
export portable state
recover from USB
propose memory updates
prepare release artifacts

```

Those actions need a stable mode language.

Without this profile, a worker or implementation agent may make the dangerous simplification:

```

AB_MODE=B -> allowed to write / run / publish / ingest / remember

```

That simplification is false.

This profile defines:

1. global A/B semantics;
2. dry-run vs apply semantics;
3. enforce vs apply semantics;
4. confirm phrase classes;
5. arm and second-arm rules;
6. runner and persistent gate rules;
7. local exception declaration;
8. witness and ledger requirements;
9. fail-closed behavior;
10. conformance checks;
11. integration requirements for the future local checker and schema validator.

2. Non-goals

This profile does not define or permit:

1. broad autonomous execution;
2. unattended persistent worker loops without separate persistent gate;
3. agent self-authorization;
4. direct memory writes by agents;
5. witness tampering;
6. public release without release gate;
7. incident repair before evidence preservation;
8. cloud upload of restricted material by AB=B;
9. hack-back;

10. live external counter-operation;
11. malware behavior;
12. unauthorized scanning;
13. credential extraction;
14. covert persistence;
15. using a confirm phrase as a secret, password, legal consent, or human judgment substitute.

A passing gate may authorize a bounded transition.

It does not create sovereignty.

3. Corpus bridge set

3.1 Explicit bridge: $c = a + b$

In $c = a + b$, CLI agents and gate logic belong to b : tools, procedures, code, ledgers, validators, scripts, models, and execution paths.

A/B modes regulate how parts of b may act under c governance.

They do not replace c .

They do not replace the human anchor a .

They do not authorize a worker to become will, memory, judge, or release authority.

3.2 Quiet bridge I: information theory

A mode flag is a compressed signal. If the same signal means “dry-run” in one module, “apply” in another, and “disable” in a third, then the system leaks ambiguity into execution. This profile adds redundancy: mode declaration, apply flag, confirm phrase, arm flags, capability checks, sandbox checks, and witness records must agree before durable state changes occur.

3.3 Quiet bridge II: cybernetics

A control loop needs clear states and negative feedback. A allows observation without committing. B allows controlled action only after the error-checking gates are satisfied. A gate failure is not an inconvenience; it is the braking system doing its job.

3.4 Earth paragraph

An electrical panel cannot have one breaker labelled “safe” and another labelled “also safe, probably”. On a real job, the difference between “test circuit”, “energized circuit”, “locked-out circuit”, and “authorized live work” matters. Same here: `dry_run`,

apply, enforce, runner, and persistent are different states. Mixing them is how you burn the wall and then explain to the insurance company that “the label felt intuitive”.

4. Core terminology

Term	Definition
A	Global observe / dry-run / plan mode. Durable mutation is forbidden unless a local read-only record explicitly declares otherwise and the operation writes only non-sensitive diagnostic output approved by this profile.
B	Global controlled apply / enforce mode. Durable mutation may occur only after task, permission, sandbox, gate, confirm, witness, and review requirements pass.
AB_MODE	A global or module-level environment flag. Its value alone never grants authority.
Local AB flag	A module-specific flag such as <code>ESTER_*_AB</code> , <code>*_AB</code> , or equivalent. It must declare whether it follows global semantics or has local exception semantics.
dry_run	Execution path that computes and reports intended actions but writes no durable operational state, no memory, no inbox item, no request object, no release artifact, and no irreversible marker.
apply	Operator-requested transition from planning to durable action. <code>apply=true</code> only asks for a gate evaluation; it does not pass the gate by itself.
enforce	Guard behavior that actively blocks, holds, revokes, freezes, quarantines, or refuses unsafe transitions. Enforcement may write minimal witness/ledger state when required by policy.
Confirm phrase	Exact explicit string or equivalent local confirmation object required by a gate class. It is a friction and intent marker, not a secret and not a substitute for task contract, permission, or review.
Arm flag	Environment/configuration condition that enables a gate path. Example class: <code>FEATURE=1</code> plus <code>FEATURE_ARMED=1</code> .
Second arm	Additional arm condition required for higher-risk subpaths such as report observation, runner execution, persistent mode, or special coordination.
Runner gate	Gate that permits a worker runner to execute queued work. It is stronger than ordinary apply.
Persistent gate	Gate that permits a long-running or repeated loop. It is stronger than a one-cycle apply.
Gate failure	Any missing, inconsistent, stale, unsafe, or prohibited gate condition. Must resolve to deny/hold/quarantine/fail-closed, not best-effort execution.
Local exception	A declared module whose local A/B flag does not follow global semantics. The exception must be explicit, documented, tested, and not generalized.

5. Global AB semantics

5.1 Mode A: observe / dry-run / plan

Mode A means:

```
inspect only
plan only
simulate only
report intended actions
validate shape
compute diffs
show candidate changes
write no durable operational state
```

Allowed in A:

```
read-only status
read-only scan
dry-run validation
diff preview
package plan
fixture preview
schema lint without writing results
report of what would happen
```

Forbidden in A:

```
create durable request queue item
promote mailbox item
write memory
write witness as if action happened
write release artifacts
run worker command with side effects
modify repository files
change branch state
mark quarantine clear
delete or move evidence
publish or deploy
start persistent daemon
```

A module claiming `dry_run=true` must prove that it writes nothing outside explicitly allowed ephemeral diagnostic output.

5.2 Mode B: controlled apply / enforce

Mode B means:

```
controlled apply may be considered
```

It does not mean:

```
permission granted
scope approved
review complete
memory promotion allowed
public release allowed
runner execution allowed
persistent loop allowed
```


Allowed in B only after all relevant gates pass:

```
write bounded ledger event
promote already accepted quarantined handoff to local inbox
enqueue a non-auto-executing request
create controlled package artifact
write evidence sidecar metadata
mark baseline seen
freeze or quarantine unsafe path
run local checker
execute bounded runner in approved sandbox
```

Still forbidden in B without separate gates:

```
direct memory write
public release
production deployment
persistent daemon
cloud upload of restricted material
incident repair before preservation
runner execution without runner gate
quarantine clear without evidence packet
self-approval
external live action
```

6. Gate hierarchy

A CGAM implementation must treat gates as layered.

```
G0: read-only observation
G1: dry-run planning
G2: controlled local apply
G3: witness / ledger write
G4: mailbox promotion / handoff enqueue
G5: runner execution
G6: persistent / repeated operation
G7: memory promotion
G8: release / publication
G9: incident clear / evidence custody transition
G10: identity / continuity / core mutation
```

6.1 Gate precedence

Higher gates inherit lower gates.

```
G5 runner execution requires:
  G0/G1/G2 validity
  task contract
  permission grant
  sandbox/worktree boundary
  capability availability
  runner confirm
  runner arm
  read-only or approved bounded sandbox
  witness/ledger path
```

```
G6 persistent operation requires:
  separate persistent enable flag
  separate persistent arm flag
  persistent confirm
```

bounded max-cycle / sleep / stop condition
 no hidden runner coupling unless explicitly profiled
 kill switch / operator interrupt path

G7 memory promotion requires:
 memory gate profile
 source linkage
 uncertainty marking
 reviewer decision
 witness reference
 no raw evidence ingestion

G8 release/publication requires:
 release readiness gate
 public/restricted split
 redaction check
 artifact inventory
 README/index discoverability
 no stale status claims
 no restricted raw evidence

G9 incident clear requires:
 evidence packet
 sidecar reference
 hash verification
 reviewer/human gate if required
 quarantine clear decision

G10 identity/core mutation is outside ordinary CLI-agent authority.
 It requires explicit human anchor and `c` governance gate.

7. Confirm phrase classes

Confirm phrases must be classed by gate type.

The same confirmation text should not silently authorize unrelated stronger transitions.

Class	Gate class	Meaning	Minimum extra requirements
CP-STATUS	G0	Read-only status; normally no confirm needed.	Must remain dry-run.
CP-APPLY	G2/G3	Controlled local apply or minimal ledger write.	Task scope, permission, arm flag if configured.
CP-BASELINE	G3/G4	Mark existing items as seen/baselined.	No enqueue of old items; witness/ledger.
CP-MAILBOX	G4	Promote quarantined mailbox item.	Quarantine validation, transfer hash, no auto-execute.
CP-RUNNER	G5	Execute queued bounded worker.	Runner arm, capability, sandbox, task contract, no self-approval.

Class	Gate class	Meaning	Minimum extra requirements
CP-PERSISTENT	G6	Run repeated/persistent loop.	Persistent arm, cycle budget, kill switch, no unauthorized runner coupling.
CP-MEMORY	G7	Promote output into memory.	Memory gate, review, source linkage, uncertainty.
CP-RELEASE	G8	Public/release artifact transition.	Release gate, redaction gate, artifact inventory.
CP-EVIDENCE	G9	Evidence custody / clear / incident transition.	Sidecar, hash, retention, reviewer/human gate.
CP-CORE	G10	Identity/continuity/core mutation.	Not available to ordinary CLI agents.

Implementation may use concrete phrase constants.

The normative requirement is not the literal string.

The normative requirement is:

```
explicit gate class
explicit confirm requirement
exact match
no implicit promotion to stronger gate
no reuse across incompatible transition classes without declaration
```

8. Arm and second-arm rules

A mode flag is not enough.

A safe gate may require all of the following:

```
feature_enable == true
feature_armed == true
subfeature_armed == true
capability_available == true
sandbox_class acceptable
confirm_phrase exact
task_contract valid
permission_grant valid
memory/off or auto_ingest=false if required
persistent disabled unless explicitly gated
runner disabled unless explicitly gated
```

8.1 Required fail-closed cases

The implementation must fail closed when:

```
apply=true but confirm missing
apply=true but enable flag missing
apply=true but arm flag missing
runner=true but runner_arm missing
runner=true but capability unavailable
runner=true but sandbox is too broad
```

```

persistent=true but persistent_arm missing
persistent=true and runner=true without explicit profile
auto_ingest=true by default
memory not explicitly off for handoff/transfer layers
SISTER_AUTOCHAT or equivalent automatic chat loop is enabled in a Codex daemon gate

```

8.2 Why second arms exist

Second arms prevent accidental escalation from:

```

observe reports
-> promote reports
-> enqueue handoff
-> run worker
-> persistent loop

```

Each arrow is a separate authority transition.

A single B flag must not unlock the whole chain.

9. Local AB exception rules

9.1 Default rule

Every local *_AB or AB_MODE flag must be classified as one of:

```

global_semantics
observe_only
apply_only_after_gate
noop_on_B
inverted_local
read_only_same_in_A_and_B
deprecated
unknown

```

If classification is missing:

```

classification = unknown
effective_mode = A
action = deny_apply

```

9.2 Local inversion rule

If a local module uses B to disable a path, or uses A/B for something other than global dry-run/apply semantics, the module must include a local exception record.

Example pattern:

```

schema: cli-agent-local-ab-exception-0.1
module: modules.media.rag_sink
flag: ESTER_RAG_INGEST_AB
classification: noop_on_B
global_semantics_followed: false
local_rule: "B disables ingestion / returns no-op"
allowed_reason: "safety switch for RAG ingestion path"

```

```
may_be_generalized: false
checker_action: "warn_and_require_explicit_mapping"
```

This prevents an implementation worker from assuming:

```
B always means apply
```

when a local module means:

```
B means no-op / disabled / special guard
```

9.3 Read-only same-mode rule

Some modules may behave the same in A and B because they only inspect or probe.

That is allowed only if the operation is truly read-only.

Example classification:

```
classification: read_only_same_in_A_and_B
durable_mutation: false
network_side_effects: none_or_probe_only
memory_write: false
requires_confirm: false
```

9.4 Deprecated or unknown flags

Unknown local AB flags must be treated as unsafe for implementation handoff.

The local checker should emit:

```
AB_UNKNOWN_FLAG
AB_UNDECLARED_LOCAL_SEMANTICS
AB_INVERSION_NOT_DECLARED
AB_B_USED_AS_AUTHORITY
```

10. Required object records

10.1 CLI_AGENT_AB_MODE_DECLARATION

```
schema: cli-agent-ab-mode-declaration-0.1
mode_id: "global-ab-mode"
scope: "global"
source: "environment|config|task_contract|module_default"
flag_name: "AB_MODE"
raw_value: "A|B|other"
normalized_value: "A|B"
unknown_value_policy: "coerce_to_A"
effective_semantics:
  A: "observe_dry_run_no_durable_mutation"
  B: "controlled_apply_only_after_gates"
declared_by: "c|operator|config|local_checker"
created_at: "2026-05-17T00:00:00Z"
```

10.2 CLI_AGENT_GATE_SEMANTICS_RECORD

```

schema: cli-agent-gate-semantics-record-0.1
gate_id: "gate.codex_daemon.runner"
gate_class: "G5"
operation: "runner_execution"
mode_required: "B"
dry_run_supported: true
apply_supported: true
confirm_class: "CP-RUNNER"
required_arms:
  - "SYNAPS_CODEX_DAEMON"
  - "SYNAPS_CODEX_DAEMON_ARMED"
  - "SYNAPS_CODEX_DAEMON_RUNNER"
  - "SYNAPS_CODEX_DAEMON_RUNNER_ARMED"
required_capabilities:
  - "codex_worker_available"
sandbox_requirement: "read_only_or_profiled_bounded"
memory_default: "off"
auto_ingest_default: false
witness_required: true
persistent_allowed: false
fail_closed_on_missing_condition: true

```

10.3 CLI_AGENT_CONFIRM_PHRASE_RECORD

```

schema: cli-agent-confirm-phrase-record-0.1
confirm_id: "confirm.codex_daemon.runner"
confirm_class: "CP-RUNNER"
phrase_source: "implementation_constant|operator_input|sealed_config"
literal_phrase_public: false
exact_match_required: true
scope:
  gate_classes: ["G5"]
  operations: ["runner_execution"]
not_valid_for:
  - "persistent_daemon"
  - "memory_promotion"
  - "public_release"
  - "identity_core_mutation"
rotation_policy: "implementation_defined"
log_policy: "record_confirm_class_not_secret_value"

```

10.4 CLI_AGENT_GATE_EVALUATION_RECORD

```

schema: cli-agent-gate-evaluation-record-0.1
evaluation_id: "gateeval.2026-05-17.example"
operation: "promote_mailbox"
gate_class: "G4"
requested_mode: "B"
dry_run: false
apply: true
confirm_class: "CP-MAILBOX"
confirm_present: true
required_arms_present: true
capability_present: true
sandbox_ok: true
task_contract_id: "catc.example"
permission_grant_id: "perm.example"
memory: "off"
auto_ingest: false
decision: "allow|deny|hold|quarantine|fail_closed"
problems: []
witness_event_id: "we.example"
created_at: "2026-05-17T00:00:00Z"

```

10.5 CLI_AGENT_LOCAL_AB_EXCEPTION_RECORD

```

schema: cli-agent-local-ab-exception-record-0.1
module: "modules.example"
flag: "EXAMPLE_AB"
classification: "global_semantics|observe_only|apply_only_after_gate|noop_on_B|inverted_local|
read_only_same_in_A_and_B|deprecated|unknown"
global_semantics_followed: true
durable_mutation_possible: false
memory_write_possible: false
network_side_effect_possible: false
confirm_required_if_apply: false
reason: "short human-readable explanation"
checker_action: "allow|warn|deny_apply|require_profile"
last_reviewed_at: "2026-05-17"

```

11. Witness, memory, and evidence interaction

11.1 Witness

Gate transitions above G2 should be witnessable.

A witness event records:

```

boundary crossed
gate evaluated
decision made
minimal reason code
reference to evidence packet or sidecar if needed

```

A witness event must not become:

```

raw evidence dump
secret store
full prompt archive
private memory export
legal evidence conclusion

```

11.2 Memory

AB=B never permits direct memory write.

Memory promotion requires:

```

memory gate
source linkage
uncertainty marking
review
witness reference
no raw evidence ingestion
no direct agent self-promotion

```

For handoff and transfer paths, default must remain:

```

memory: "off"
auto_ingest: false

```

11.3 Raw evidence

Evidence custody transitions are not ordinary apply actions.

They require:

```
raw evidence sidecar profile
hash or signature reference
retention class
access policy
reviewer / human gate if needed
redaction boundary
```

12. Implementation mapping to SYNAPS / Codex bridge

The current implementation substrate already demonstrates a useful gate vocabulary.

The CGAM profile should map it as follows:

Implementation pattern	CGAM interpretation
<code>dry_run: true</code>	G0/G1 observation; no durable mutation.
<code>apply: true</code>	Request gate evaluation for durable transition.
<code>confirm_required</code>	Confirm phrase class required by gate.
<code>*_ARMED</code>	Arm / second-arm requirement.
<code>RUNNER + RUNNER_ARMED</code>	G5 runner execution gate.
<code>PERSISTENT + PERSISTENT_ARMED</code>	G6 persistent loop gate.
<code>sandbox=read-only</code>	Runner-safe or reduced side-effect boundary, depending on task.
<code>auto_execute=false</code>	Handoff may be queued without worker execution.
<code>memory=off</code>	Handoff/transfer must not become memory by default.
<code>auto_ingest=false</code>	Received material must not enter memory automatically.
<code>events.jsonl / ledger</code>	Witness or operational event surface, not raw evidence.

This mapping is normative for future CGAM documentation.

It is not a claim that the current code is complete conformance.

13. Conformance rules

AB-CONF-001 — Unknown mode fails to A

Input:

```
raw_value: "C"
```


Expected:

```
normalized_value: "A"
decision: "deny_apply"
problem: "unknown_ab_mode"
```

AB-CONF-002 — Dry-run writes nothing

A dry-run operation may report planned actions.

It must not create:

```
inbox item
request object
release artifact
memory record
quarantine-clear marker
durable apply ledger claiming action occurred
```

AB-CONF-003 — Apply without confirm fails closed

Input:

```
apply: true
confirm_present: false
```

Expected:

```
decision: "fail_closed"
problem: "missing_confirm_phrase"
```

AB-CONF-004 — Runner requires stronger gate

Runner execution requires:

```
runner enable
runner arm
runner confirm class
capability availability
sandbox requirement
task contract
permission grant
witness/ledger path
```

Missing any item:

```
decision: "fail_closed"
```

AB-CONF-005 — Persistent requires separate persistent gate

Persistent mode requires:

```
persistent enable
persistent arm
persistent confirm
bounded cycle policy
```

```
kill switch
no hidden runner coupling
```

Missing any item:

```
decision: "fail_closed"
```

AB-CONF-006 — Persistent and runner must not silently combine

If persistent mode and runner mode are both active, the default decision is:

```
decision: "fail_closed"
problem: "runner_must_remain_disabled_for_persistent"
```

unless an explicit future profile safely defines the combined mode.

AB-CONF-007 — Local inversion must be declared

If a local *_AB flag uses B as no-op, disable, or inverse behavior, a CLI_AGENT_LOCAL_AB_EXCEPTION_RECORD is required.

Missing record:

```
decision: "deny_apply"
problem: "ab_inversion_not_declared"
```

AB-CONF-008 — Memory and auto-ingest defaults are off

For handoff, transfer, mailbox, package ledger, report observer, peer activity, and worker queue objects:

```
memory: "off"
auto_ingest: false
```

must be the default.

Any default memory=on or auto_ingest=true is a red-line failure unless separately justified by a memory-gate profile and explicit task contract.

AB-CONF-009 — Confirm phrase class cannot be upgraded silently

A CP-APPLY confirmation cannot authorize:

```
runner execution
persistent daemon
memory promotion
release publication
incident evidence clear
identity/core mutation
```

AB-CONF-010 — B does not bypass public/restricted split

Mode B cannot publish restricted content.

Release/publication requires release gate and redaction gate.

14. Red-line failures

The following invalidate implementation-readiness or conformance claims:

```
AB=B treated as sufficient authority
dry_run path writes durable operational state
apply path proceeds without confirm where confirm is required
runner path executes without runner arm
persistent path starts without persistent arm
persistent path combines with runner without explicit profile
local AB inversion undocumented
memory writes happen directly from CLI agent output
auto_ingest defaults to true in handoff/transfer paths
confirm phrase logs expose secret/sealed values
one confirm phrase authorizes unrelated stronger gates
release/publication uses AB=B instead of release gate
incident clear uses AB=B instead of evidence sidecar + review
unknown mode is treated as B
gate failure is downgraded to warning
```

15. Required integration with Local Checker

The future `CLI_Agent_Local_Checker_Profile_v0_1.md` must implement checks for:

```
AB_MODE parser
local AB flag inventory
local exception records
dry-run no-write assertions
apply requires confirm
runner requires runner gate
persistent requires persistent gate
persistent+runner default denial
memory=off defaults
auto_ingest=false defaults
confirm class separation
release gate separation
evidence sidecar separation
```

Minimum local checker outputs:

```
AB_OK
AB_WARNING
AB_DENY_APPLY
AB_FAIL_CLOSED
AB_UNDECLARED_LOCAL_EXCEPTION
AB_DRY_RUN_MUTATION
AB_CONFIRM_CLASS_COLLISION
AB_MEMORY_DEFAULT_UNSAFE
AB_PERSISTENT_RUNNER_COLLISION
```

16. Required integration with Schema Object Registry

The future schema registry must include:

```
cli-agent-ab-mode-declaration-0.1.schema.json
cli-agent-gate-semantics-record-0.1.schema.json
cli-agent-confirm-phrase-record-0.1.schema.json
cli-agent-arm-status-record-0.1.schema.json
cli-agent-dry-run-apply-record-0.1.schema.json
cli-agent-local-ab-exception-record-0.1.schema.json
cli-agent-gate-evaluation-record-0.1.schema.json
```

The registry must mark JSON Schema as necessary but not sufficient.

Semantic validation remains required for:

```
mode-to-gate compatibility
confirm-class compatibility
runner/persistent incompatibility
local exception legitimacy
memory/auto-ingest defaults
red-line denial
release/public split
```

17. Required integration with Open Issues and Contradiction Register

This profile should close or partially close the following issue classes:

```
AB/gate semantics ambiguity
dry-run/apply wording drift
confirm phrase class ambiguity
runner vs persistent gate ambiguity
local AB inversion risk
memory=off / auto_ingest=false default drift
```

It should not claim to close:

```
actual JSON schemas not yet extracted
local checker not yet implemented
conformance tests not yet run
provider-specific profiles not yet created
release/public surface profile not yet completed
```

18. Public redaction boundary

This profile is public-safe if it avoids:

```
literal secret values
private tokens
real infrastructure credentials
real incident gate details
```

```
private operator paths
live defensive signatures
restricted sidecar content
```

Confirm phrase **class names** may be public.

Concrete literal confirm phrase values should be treated as implementation details unless already intentionally public in code. Even when public, they must not be described as secrets.

A confirm phrase is a deliberate-friction gate.

It is not authentication.

19. Status claims allowed by this profile

Allowed after this profile exists:

```
AB/gate semantics have a draft normative profile.
Global A/B semantics are defined.
Local AB exceptions are recognized as an implementation risk.
Runner and persistent gates are separated at the profile level.
Mode flags are not treated as sufficient authority.
```

Not allowed until implementation and tests exist:

```
AB semantics are fully implemented.
All local AB flags are inventoried.
Local checker enforces this profile.
Conformance tests passed.
Persistent/runner gates are verified in production.
Release is implementation-complete.
```

20. Minimal implementation-readiness checklist

Before Codex implementation handoff, the package must include:

```
[ ] This profile in package index.
[ ] Glossary updated with A/B, dry_run, apply, enforce, confirm phrase, arm, second arm, runner gate,
persistent gate.
[ ] Open Issues updated.
[ ] Contradiction Register updated.
[ ] Schema extraction plan updated with AB object family.
[ ] Local Checker Profile includes AB checks.
[ ] Fixture pack includes AB dry-run/apply/runner/persistent cases.
[ ] Release notes state that AB profile exists but enforcement is pending until checker/tests run.
```

21. Canonical short rules

A means observe.
B means controlled apply may be evaluated.
B alone authorizes nothing.
Apply requires a gate.
Runner requires a stronger gate.
Persistent requires a separate stronger gate.
Dry-run writes nothing.
Memory stays off by default.
Auto-ingest stays false by default.
Unknown means A / deny apply.
Local inversions must be declared.
Gate failure is a safety signal, not a suggestion.

22. Closing note

This profile is intentionally small compared with the rest of the CGAM package.

That is its function.

It labels the switches.

Without it, the implementation can have correct pieces and still behave incorrectly because the same flag is interpreted in three different ways.

The next implementation-facing documents should treat this profile as a hard dependency:

Registry Profile
Local Checker Profile
Schema Object Registry
Semantic Validator Rules
Conformance Evidence Packet Profile
v0.1.1 Hygiene Patch Notes